

SOFTWARE FOUNDATIONS

VOLUME 1: LOGICAL FOUNDATIONS

[TABLE OF CONTENTS](#)[INDEX](#)[ROADMAP](#)

INDUCTION

PROOF BY INDUCTION

Separate Compilation

Before getting started on this chapter, we need to import all of our definitions from the previous chapter:

```
From LF Require Export Basics.
```

For this `Require` command to work, Coq needs to be able to find a compiled version of the previous chapter (`Basics.v`). This compiled version, called `Basics.vo`, is analogous to the `.class` files compiled from `.java` source files and the `.o` files compiled from `.c` files.

To compile `Basics.v` and obtain `Basics.vo`, first make sure that the files `Basics.v`, `Induction.v`, and `_CoqProject` are in the current directory.

The `_CoqProject` file should contain just the following line:

```
-Q . LF
```

This maps the current directory (`"."`, which contains `Basics.v`, `Induction.v`, etc.) to the prefix (or "logical directory") `"LF"`. Proof General, CoqIDE, and VSCoq read `_CoqProject` automatically, to find out to where to look for the file `Basics.vo` corresponding to the library `LF.Basics`.

Once the files are in place, there are various ways to build `Basics.vo` from an IDE, or you can build it from the command line. From an IDE...

- In Proof General: The compilation can be made to happen automatically when you submit the `Require` line above to PG, by setting the emacs variable `coq-compile-before-require` to `t`. This can also be found in the menu: `"Coq" > "Auto Compilation" > "Compile Before Require"`.
- In CoqIDE: One thing you can do on all platforms is open `Basics.v`; then, in the `"Compile"` menu, click on `"Compile Buffer"`.
- For VSCode users, open the terminal pane at the bottom and then follow the command line instructions below. (If you downloaded the project setup

.tgz file, just doing ``make`` should build all the code.)

To compile `Basics.v` from the command line...

- First, generate a Makefile using the `coq_makefile` utility, which comes installed with Coq. (If you obtained the whole volume as a single archive, a Makefile should already exist and you can skip this step.)

```
coq_makefile -f _CoqProject *.v -o Makefile
```

You should rerun that command whenever you add or remove Coq files in this directory.

- Now you can compile `Basics.v` by running `make` with the corresponding `.vo` file as a target:

```
make Basics.vo
```

All files in the directory can be compiled by giving no arguments:

```
make
```

- Under the hood, `make` uses the Coq compiler, `coqc`. You can also run `coqc` directly:

```
coqc -Q . LF Basics.v
```

- Since `make` also calculates dependencies between source files to compile them in the right order, `make` should generally be preferred over running `coqc` explicitly. But as a last (but not terrible) resort, you can simply compile each file manually as you go. For example, before starting work on the present chapter, you would need to run the following command:

```
coqc -Q . LF Basics.v
```

Then, once you've finished this chapter, you'd do

```
coqc -Q . LF Induction.v
```

to get ready to work on the next one. If you ever remove the `.vo` files, you'd need to give both commands again (in that order).

Troubleshooting:

- For many of the alternatives above you need to make sure that the `coqc` executable is in your `PATH`.
- If you get complaints about missing identifiers, it may be because the "load path" for Coq is not set up correctly. The `Print LoadPath.` command may be helpful in sorting out such issues.
- When trying to compile a later chapter, if you see a message like


```
Compiled library Induction makes inconsistent assumptions over
library Basics
```

 a common reason is that the library `Basics` was modified and recompiled without also recompiling `Induction` which depends on it. Recompile `Induction`, or everything if too many files are affected (for instance by running `make` and if even this doesn't work then `make clean; make`).
- If you get complaints about missing identifiers later in this file it may be because the "load path" for Coq is not set up correctly. The `Print`

`LoadPath.` command may be helpful in sorting out such issues.

In particular, if you see a message like

```
Compiled library Foo makes inconsistent assumptions over
library Bar
```

check whether you have multiple installations of Coq on your machine. It may be that commands (like `coqc`) that you execute in a terminal window are getting a different version of Coq than commands executed by Proof General or CoqIDE.

- One more tip for CoqIDE users: If you see messages like `Error: Unable to locate library Basics`, a likely reason is inconsistencies between compiling things *within CoqIDE* vs *using coqc from the command line*. This typically happens when there are two incompatible versions of `coqc` installed on your system (one associated with CoqIDE, and one associated with `coqc` from the terminal). The workaround for this situation is compiling using CoqIDE only (i.e. choosing "make" from the menu), and avoiding using `coqc` directly at all.

Proof by Induction

We can prove that `0` is a neutral element for `+` on the *left* using just reflexivity. But the proof that it is also a neutral element on the *right* ...

```
Theorem add_0_r_firsttry : ∀ n:nat,
  n + 0 = n.
```

... can't be done in the same simple way. Just applying reflexivity doesn't work, since the `n` in `n + 0` is an arbitrary unknown number, so the match in the definition of `+` can't be simplified.

```
Proof.
  intros n.
  simpl. (* Does nothing! *)
Abort.
```

And reasoning by cases using `destruct n` doesn't get us much further: the branch of the case analysis where we assume `n = 0` goes through fine, but in the branch where `n = S n'` for some `n'` we get stuck in exactly the same way.

```
Theorem add_0_r_secondtry : ∀ n:nat,
  n + 0 = n.
Proof.
  intros n. destruct n as [| n'] eqn:E.
  - (* n = 0 *)
    reflexivity. (* so far so good... *)
  - (* n = S n' *)
    simpl. (* ...but here we are stuck again *)
Abort.
```

We could use `destruct n'` to get one step further, but, since `n` can be arbitrarily large, we'll never get all the there if we just go on like this.

To prove interesting facts about numbers, lists, and other inductively defined sets, we often need a more powerful reasoning principle: *induction*.

Recall (from a discrete math course, probably) the *principle of induction over natural numbers*: If $P(n)$ is some proposition involving a natural number n and we want to show that P holds for all numbers n , we can reason like this:

- show that $P(0)$ holds;
- show that, for any n' , if $P(n')$ holds, then so does $P(S\ n')$;
- conclude that $P(n)$ holds for all n .

In Coq, the steps are the same: we begin with the goal of proving $P(n)$ for all n and break it down (by applying the `induction` tactic) into two separate subgoals: one where we must show $P(0)$ and another where we must show $P(n') \rightarrow P(S\ n')$. Here's how this works for the theorem at hand:

```
Theorem add_0_r : ∀ n:nat, n + 0 = n.
```

```
Proof.
```

```
  intros n. induction n as [| n' IHn'].
```

```
  - (* n = 0 *) reflexivity.
```

```
  - (* n = S n' *) simpl. rewrite → IHn'. reflexivity. Qed.
```

Like `destruct`, the `induction` tactic takes an `as...` clause that specifies the names of the variables to be introduced in the subgoals. Since there are two subgoals, the `as...` clause has two parts, separated by `|`. (Strictly speaking, we can omit the `as...` clause and Coq will choose names for us. In practice, this is a bad idea, as Coq's automatic choices tend to be confusing.)

In the first subgoal, n is replaced by 0 . No new variables are introduced (so the first part of the `as...` is empty), and the goal becomes $0 = 0 + 0$, which follows by simplification.

In the second subgoal, n is replaced by $S\ n'$, and the assumption $n' + 0 = n'$ is added to the context with the name `IHn'` (i.e., the Induction Hypothesis for n'). These two names are specified in the second part of the `as...` clause. The goal in this case becomes $S\ n' = (S\ n') + 0$, which simplifies to $S\ n' = S\ (n' + 0)$, which in turn follows from `IHn'`.

```
Theorem minus_n_n : ∀ n,
```

```
  minus n n = 0.
```

```
Proof.
```

```
  (* WORKED IN CLASS *)
```

```
  intros n. induction n as [| n' IHn'].
```

```
  - (* n = 0 *)
```

```
    simpl. reflexivity.
```

```
  - (* n = S n' *)
```

```
    simpl. rewrite → IHn'. reflexivity. Qed.
```

(The use of the `intros` tactic in these proofs is actually redundant. When applied to a goal that contains quantified variables, the `induction` tactic will automatically move them into the context as needed.)

Exercise: 2 stars, standard, especially useful (basic induction)

Prove the following using induction. You might need previously proven results.

```

Theorem mul_0_r : ∀ n:nat,
  n × 0 = 0.
Proof.
  (* FILL IN HERE *) Admitted.

Theorem plus_n_Sm : ∀ n m : nat,
  S (n + m) = n + (S m).
Proof.
  (* FILL IN HERE *) Admitted.

Theorem add_comm : ∀ n m : nat,
  n + m = m + n.
Proof.
  (* FILL IN HERE *) Admitted.

Theorem add_assoc : ∀ n m p : nat,
  n + (m + p) = (n + m) + p.
Proof.
  (* FILL IN HERE *) Admitted.

```

□

Exercise: 2 stars, standard (double_plus)

Consider the following function, which doubles its argument:

```

Fixpoint double (n:nat) :=
  match n with
  | 0 ⇒ 0
  | S n' ⇒ S (S (double n'))
  end.

```

Use induction to prove this simple fact about double:

```

Lemma double_plus : ∀ n, double n = n + n .
Proof.
  (* FILL IN HERE *) Admitted.

```

□

Exercise: 2 stars, standard (eqb_refl)

The following theorem relates the computational equality `=?` on `nat` with the definitional equality `=` on `bool`.

```

Theorem eqb_refl : ∀ n : nat,
  (n =? n) = true.
Proof.
  (* FILL IN HERE *) Admitted.

```

□

Exercise: 2 stars, standard, optional (even_S)

One inconvenient aspect of our definition of even `n` is the recursive call on `n - 2`. This makes proofs about even `n` harder when done by induction on `n`, since we may need an induction hypothesis about `n - 2`. The following lemma gives an alternative characterization of even `(S n)` that works better with induction:

```

Theorem even_S : ∀ n : nat,
  even (S n) = negb (even n).
Proof.

```

(* FILL IN HERE *) Admitted.

□

Proofs Within Proofs

In Coq, as in informal mathematics, large proofs are often broken into a sequence of theorems, with later proofs referring to earlier theorems. But sometimes a proof will involve some miscellaneous fact that is too trivial and of too little general interest to bother giving it its own top-level name. In such cases, it is convenient to be able to simply state and prove the needed "sub-theorem" right at the point where it is used. The `assert` tactic allows us to do this.

```
Theorem mult_0_plus' : ∀ n m : nat,
  (n + 0 + 0) × m = n × m.
Proof.
  intros n m.
  assert (H: n + 0 + 0 = n).
  { rewrite add_comm. simpl. rewrite add_comm. reflexivity. }
  rewrite → H.
  reflexivity. Qed.
```

The `assert` tactic introduces two sub-goals. The first is the assertion itself; by prefixing it with `H:` we name the assertion `H`. (We can also name the assertion with `as` just as we did above with `destruct` and `induction`, i.e., `assert (n + 0 + 0 = n) as H`.) Note that we surround the proof of this assertion with curly braces `{ ... }`, both for readability and so that, when using Coq interactively, we can see more easily when we have finished this sub-proof. The second goal is the same as the one at the point where we invoke `assert` except that, in the context, we now have the assumption `H` that `n + 0 + 0 = n`. That is, `assert` generates one subgoal where we must prove the asserted fact and a second subgoal where we can use the asserted fact to make progress on whatever we were trying to prove in the first place.

As another example, suppose we want to prove that $(n + m) + (p + q) = (m + n) + (p + q)$. The only difference between the two sides of the `=` is that the arguments `m` and `n` to the first inner `+` are swapped, so it seems we should be able to use the commutativity of addition (`add_comm`) to rewrite one into the other. However, the `rewrite` tactic is not very smart about *where* it applies the rewrite. There are three uses of `+` here, and it turns out that doing `rewrite → add_comm` will affect only the *outer* one...

```
Theorem plus_rearrange_firsttry : ∀ n m p q : nat,
  (n + m) + (p + q) = (m + n) + (p + q).
Proof.
  intros n m p q.
  (* We just need to swap (n + m) for (m + n)... seems
     like add_comm should do the trick! *)
  rewrite add_comm.
  (* Doesn't work... Coq rewrites the wrong plus! :-( *)
  Abort.
```

To use `add_comm` at the point where we need it, we can introduce a local lemma stating that $n + m = m + n$ (for the *particular* m and n that we are talking about here), prove this lemma using `add_comm`, and then use it to do the desired rewrite.

```
Theorem plus_rearrange : ∀ n m p q : nat,
  (n + m) + (p + q) = (m + n) + (p + q).
Proof.
  intros n m p q.
  assert (H: n + m = m + n).
  { rewrite add_comm. reflexivity. }
  rewrite H. reflexivity. Qed.
```

Formal vs. Informal Proof

"_Informal proofs are algorithms; formal proofs are code."

What constitutes a successful proof of a mathematical claim? The question has challenged philosophers for millennia, but a rough and ready definition could be this: A proof of a mathematical proposition P is a written (or spoken) text that instills in the reader or hearer the certainty that P is true -- an unassailable argument for the truth of P . That is, a proof is an act of communication.

Acts of communication may involve different sorts of readers. On one hand, the "reader" can be a program like Coq, in which case the "belief" that is instilled is that P can be mechanically derived from a certain set of formal logical rules, and the proof is a recipe that guides the program in checking this fact. Such recipes are *formal* proofs.

Alternatively, the reader can be a human being, in which case the proof will be written in English or some other natural language, and will thus necessarily be *informal*. Here, the criteria for success are less clearly specified. A "valid" proof is one that makes the reader believe P . But the same proof may be read by many different readers, some of whom may be convinced by a particular way of phrasing the argument, while others may not be. Some readers may be particularly pedantic, inexperienced, or just plain thick-headed; the only way to convince them will be to make the argument in painstaking detail. But other readers, more familiar in the area, may find all this detail so overwhelming that they lose the overall thread; all they want is to be told the main ideas, since it is easier for them to fill in the details for themselves than to wade through a written presentation of them. Ultimately, there is no universal standard, because there is no single way of writing an informal proof that is guaranteed to convince every conceivable reader.

In practice, however, mathematicians have developed a rich set of conventions and idioms for writing about complex mathematical objects that -- at least within a certain community -- make communication fairly reliable. The conventions of this stylized form of communication give a fairly clear standard for judging proofs good or bad.

Because we are using Coq in this course, we will be working heavily with formal proofs. But this doesn't mean we can completely forget about informal ones! Formal proofs are useful in many ways, but they are *not* very efficient ways of communicating ideas between human beings.

For example, here is a proof that addition is associative:

```
Theorem add_assoc' : ∀ n m p : nat,
  n + (m + p) = (n + m) + p.
Proof. intros n m p. induction n as [| n' IHn']. reflexivity.
  simpl. rewrite IHn'. reflexivity. Qed.
```

Coq is perfectly happy with this. For a human, however, it is difficult to make much sense of it. We can use comments and bullets to show the structure a little more clearly...

```
Theorem add_assoc'' : ∀ n m p : nat,
  n + (m + p) = (n + m) + p.
Proof.
  intros n m p. induction n as [| n' IHn'].
  - (* n = 0 *)
    reflexivity.
  - (* n = S n' *)
    simpl. rewrite IHn'. reflexivity. Qed.
```

... and if you're used to Coq you might be able to step through the tactics one after the other in your mind and imagine the state of the context and goal stack at each point, but if the proof were even a little bit more complicated this would be next to impossible.

A (pedantic) mathematician might write the proof something like this:

- *Theorem:* For any n, m and p ,

$$n + (m + p) = (n + m) + p.$$
Proof: By induction on n .
 - First, suppose $n = 0$. We must show that

$$0 + (m + p) = (0 + m) + p.$$
 This follows directly from the definition of $+$.
 - Next, suppose $n = S\ n'$, where

$$n' + (m + p) = (n' + m) + p.$$
 We must now show that

$$(S\ n') + (m + p) = ((S\ n') + m) + p.$$
 By the definition of $+$, this follows from

$$S\ (n' + (m + p)) = S\ ((n' + m) + p),$$
 which is immediate from the induction hypothesis. *Qed.*

The overall form of the proof is basically similar, and of course this is no accident: Coq has been designed so that its `induction` tactic generates the same sub-goals, in the same order, as the bullet points that a mathematician would write. But there are significant differences of detail: the formal proof is much more explicit in some ways (e.g., the use of `reflexivity`) but much less explicit in others (in particular, the "proof state" at any given point in the Coq proof is

completely implicit, whereas the informal proof reminds the reader several times where things stand).

Exercise: 2 stars, advanced, especially useful (add_comm informal)

Translate your solution for add_comm into an informal proof:

Theorem: Addition is commutative.

Proof: (* FILL IN HERE *)

```
(* Do not modify the following line: *)
Definition manual_grade_for_add_comm_informal : option (nat×string) :=
None.
□
```

Exercise: 2 stars, standard, optional (eqb_refl informal)

Write an informal proof of the following theorem, using the informal proof of add_assoc as a model. Don't just paraphrase the Coq tactics into English!

Theorem: $(n =? n) = \text{true}$ for any n .

Proof: (* FILL IN HERE *)

```
(* Do not modify the following line: *)
Definition manual_grade_for_eqb_refl_informal : option (nat×string) :=
None.
□
```

More Exercises

Exercise: 3 stars, standard, especially useful (mul_comm)

Use assert to help prove add_shuffle3. You don't need to use induction yet.

```
Theorem add_shuffle3 : ∀ n m p : nat,
  n + (m + p) = m + (n + p).
Proof.
  (* FILL IN HERE *) Admitted.
```

Now prove commutativity of multiplication. You will probably want to look for (or define and prove) a "helper" theorem to be used in the proof of this one. Hint: what is $n \times (1 + k)$?

```
Theorem mul_comm : ∀ m n : nat,
  m × n = n × m.
Proof.
  (* FILL IN HERE *) Admitted.
□
```

Exercise: 2 stars, standard, optional (plus_leb_compat I)

If a hypothesis has the form $H: P \rightarrow a = b$, then rewrite H will rewrite a to b in the goal, and add P as a new subgoal. Use that in the inductive step of this exercise.

```
Check leb.
```

```
Theorem plus_leb_compat_1 : ∀ n m p : nat,
  n <=? m = true → (p + n) <=? (p + m) = true.
Proof.
  (* FILL IN HERE *) Admitted.
□
```

Exercise: 3 stars, standard, optional (more exercises)

Take a piece of paper. For each of the following theorems, first *think* about whether (a) it can be proved using only simplification and rewriting, (b) it also requires case analysis (`destruct`), or (c) it also requires induction. Write down your prediction. Then fill in the proof. (There is no need to turn in your piece of paper; this is just to encourage you to reflect before you hack!)

```
Theorem leb_refl : ∀ n:nat,
  (n <=? n) = true.
Proof.
  (* FILL IN HERE *) Admitted.
```

```
Theorem zero_neqb_S : ∀ n:nat,
  0 =? (S n) = false.
Proof.
  (* FILL IN HERE *) Admitted.
```

```
Theorem andb_false_r : ∀ b : bool,
  andb b false = false.
Proof.
  (* FILL IN HERE *) Admitted.
```

```
Theorem S_neqb_0 : ∀ n:nat,
  (S n) =? 0 = false.
Proof.
  (* FILL IN HERE *) Admitted.
```

```
Theorem mult_1_l : ∀ n:nat, 1 × n = n.
Proof.
  (* FILL IN HERE *) Admitted.
```

```
Theorem all3_spec : ∀ b c : bool,
  orb
    (andb b c)
    (orb (negb b)
         (negb c))
  = true.
Proof.
  (* FILL IN HERE *) Admitted.
```

```
Theorem mult_plus_distr_r : ∀ n m p : nat,
  (n + m) × p = (n × p) + (m × p).
Proof.
  (* FILL IN HERE *) Admitted.
```

```
Theorem mult_assoc : ∀ n m p : nat,
  n × (m × p) = (n × m) × p.
Proof.
  (* FILL IN HERE *) Admitted.
```

□

Exercise: 2 stars, standard, optional (add_shuffle3')

The `replace` tactic allows you to specify a particular subterm to rewrite and what you want it rewritten to: `replace (t) with (u)` replaces (all copies of) expression `t` in the goal by expression `u`, and generates `t = u` as an additional subgoal. This is often useful when a plain `rewrite` acts on the wrong part of the goal.

Use the `replace` tactic to do a proof of `add_shuffle3'`, just like `add_shuffle3` but without needing `assert`.

```
Theorem add_shuffle3' : ∀ n m p : nat,
  n + (m + p) = m + (n + p).
Proof.
  (* FILL IN HERE *) Admitted.
□
```

Nat to Bin and Back to Nat

Recall the `bin` type we defined in [Basics](#):

```
Inductive bin : Type :=
| Z
| B0 (n : bin)
| B1 (n : bin)
.
```

Before you start working on the next exercise, replace the stub definitions of `incr` and `bin_to_nat`, below, with your solution from [Basics](#). That will make it possible for this file to be graded on its own.

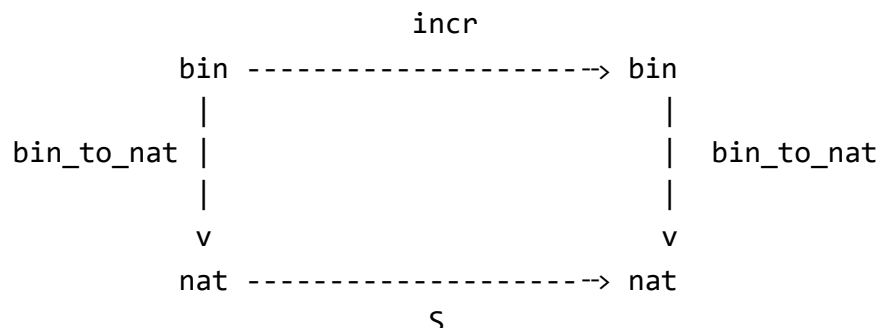
```
Fixpoint incr (m:bin) : bin
  (* REPLACE THIS LINE WITH ":= _your_definition_ ." *) . Admitted.

Fixpoint bin_to_nat (m:bin) : nat
  (* REPLACE THIS LINE WITH ":= _your_definition_ ." *) . Admitted.
```

In [Basics](#), we did some unit testing of `bin_to_nat`, but we didn't prove its correctness. Now we'll do so.

Exercise: 3 stars, standard, especially useful (binary_commute)

Prove that the following diagram commutes:



That is, incrementing a binary number and then converting it to a (unary) natural number yields the same result as first converting it to a natural number and then

incrementing.

If you want to change your previous definitions of `incr` or `bin_to_nat` to make the property easier to prove, feel free to do so!

```
Theorem bin_to_nat_pres_incr : ∀ b : bin,
  bin_to_nat (incr b) = 1 + bin_to_nat b.
Proof.
  (* FILL IN HERE *) Admitted.
□
```

Exercise: 3 stars, standard (nat_bin_nat)

Write a function to convert natural numbers to binary numbers.

```
Fixpoint nat_to_bin (n:nat) : bin
  (* REPLACE THIS LINE WITH ":= _your_definition_ ." *) Admitted.
```

Prove that, if we start with any `nat`, convert it to `bin`, and convert it back, we get the same `nat` which we started with.

Hint: This proof should go through smoothly using the previous exercise about `incr` as a lemma. If not, revisit your definitions of the functions involved and consider whether they are more complicated than necessary: the shape of a proof by induction will match the recursive structure of the program being verified, so make the recursions as simple as possible.

```
Theorem nat_bin_nat : ∀ n, bin_to_nat (nat_to_bin n) = n.
Proof.
  (* FILL IN HERE *) Admitted.
□
```

Bin to Nat and Back to Bin (Advanced)

The opposite direction -- starting with a `bin`, converting to `nat`, then converting back to `bin` -- turns out to be problematic. That is, the following theorem does not hold.

```
Theorem bin_nat_bin_fails : ∀ b, nat_to_bin (bin_to_nat b) = b.
Abort.
```

Let's explore why that theorem fails, and how to prove a modified version of it. We'll start with some lemmas that might seem unrelated, but will turn out to be relevant.

Exercise: 2 stars, advanced (double_bin)

Prove this lemma about `double`, which we defined earlier in the chapter.

```
Lemma double_incr : ∀ n : nat, double (S n) = S (S (double n)).
Proof.
  (* FILL IN HERE *) Admitted.
```

Now define a similar doubling function for `bin`.

```
Definition double_bin (b:bin) : bin
  (* REPLACE THIS LINE WITH ":= _your_definition_ ." *). Admitted.
```

Check that your function correctly doubles zero.

```
Example double_bin_zero : double_bin Z = Z.
  (* FILL IN HERE *) Admitted.
```

Prove this lemma, which corresponds to `double_incr`.

```
Lemma double_incr_bin : ∀ b,
  double_bin (incr b) = incr (double_bin b).
Proof.
  (* FILL IN HERE *) Admitted.
□
```

Let's return to our desired theorem:

```
Theorem bin_nat_bin_fails : ∀ b, nat_to_bin (bin_to_nat b) = b.
Abort.
```

The theorem fails because there are some `bin` such that we won't necessarily get back to the *original* `bin`, but instead to an "equivalent" `bin`. (We deliberately leave that notion undefined here for you to think about.)

Explain in a comment, below, why this failure occurs. Your explanation will not be graded, but it's important that you get it clear in your mind before going on to the next part. If you're stuck on this, think about alternative implementations of `double_bin` that might have failed to satisfy `double_bin_zero` yet otherwise seem correct.

```
(* FILL IN HERE *)
```

To solve that problem, we can introduce a *normalization* function that selects the simplest `bin` out of all the equivalent `bin`. Then we can prove that the conversion from `bin` to `nat` and back again produces that normalized, simplest `bin`.

Exercise: 4 stars, advanced (bin_nat_bin)

Define `normalize`. You will need to keep its definition as simple as possible for later proofs to go smoothly. Do not use `bin_to_nat` or `nat_to_bin`, but do use `double_bin`.

Hint: Structure the recursion such that it *always* reaches the end of the `bin` and process each bit only once. Do not try to "look ahead" at future bits.

```
Fixpoint normalize (b:bin) : bin
  (* REPLACE THIS LINE WITH ":= _your_definition_ ." *). Admitted.
```

It would be wise to do some `Example` proofs to check that your definition of `normalize` works the way you intend before you proceed. They won't be graded, but fill them in below.

```
(* FILL IN HERE *)
```

Finally, prove the main theorem. The inductive cases could be a bit tricky.

Hint: Start by trying to prove the main statement, see where you get stuck, and see if you can find a lemma -- perhaps requiring its own inductive proof -- that will allow the main proof to make progress. We have one lemma for the B_0 case (which also makes use of `double_incr_bin`) and another for the B_1 case.

```
Theorem bin_nat_bin : ∀ b, nat_to_bin (bin_to_nat b) = normalize b.
```

```
Proof.
```

```
  (* FILL IN HERE *) Admitted.
```

```
□
```

```
(* 2025-01-13 16:00 *)
```

Index

This page has been generated by coqdoc